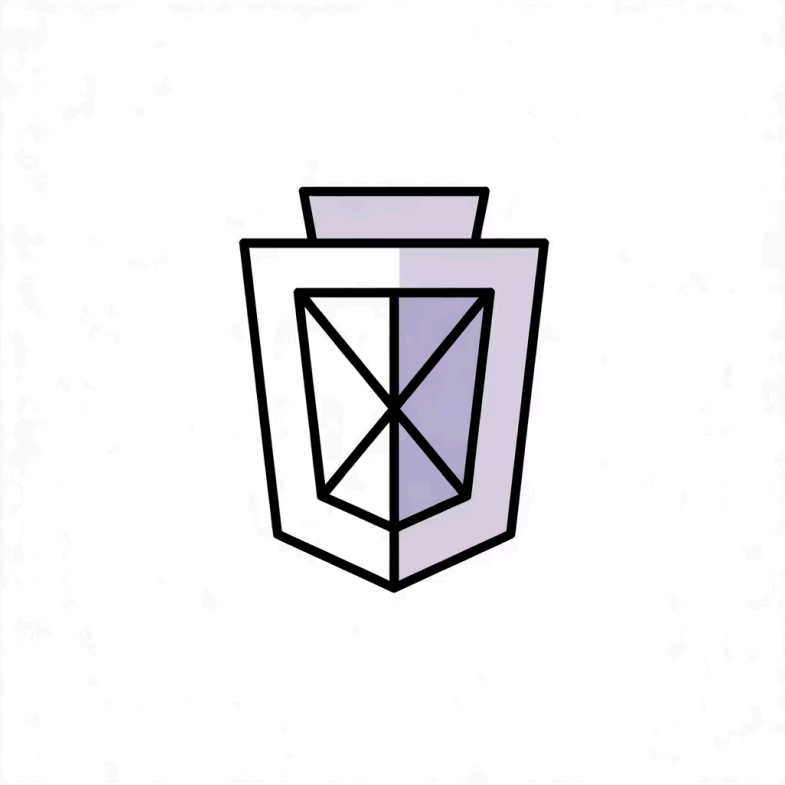
JavaScript - Jour 1

Variables, types et conditions

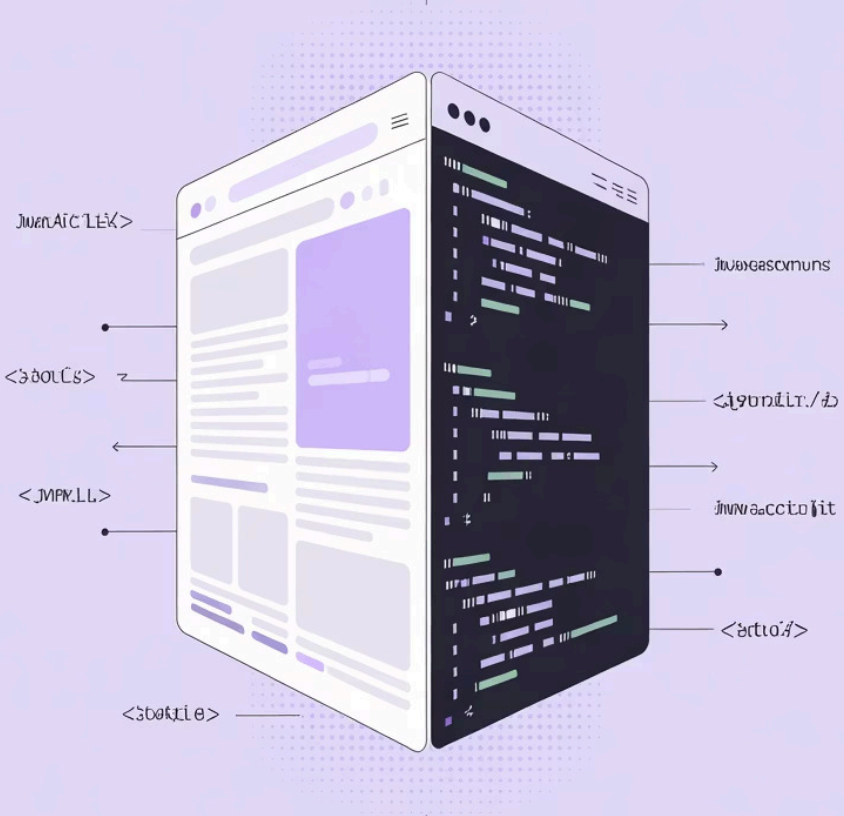


Objectifs pédagogiques de la journée

- Introduction à JavaScript
- Déclarer et utiliser des variables
- Identifier et manipuler les types de données
- Conseils sur la propreté de code
- Interagir avec l'utilisateur
- Scénarios d'exécutions

MODULE

Introduction à JavaScript



EXPLICATIONS

Qu'est-ce que JavaScript ?

JavaScript est un langage de programmation léger et puissant qui permet de rendre les pages web interactives et dynamiques.

Il est présent et utilisé sur la quasi-intégralité des sites internet.

EXPLICATIONS

Où s'exécute JavaScript ?

JavaScript est un langage polyvalent qui s'exécute dans divers environnements, chacun avec ses spécificités.

Dans les navigateurs web

Le rôle le plus connu de JavaScript est d'apporter de l'interactivité aux pages web, s'exécutant directement côté client dans le navigateur.

Applications de bureau et mobiles

Grâce à des outils comme Electron, Tauri ou React Native, JavaScript peut être utilisé pour construire des applications natives multiplateformes.

Côté serveur avec Node.js

Node.js a permis à JavaScript de s'émanciper du navigateur, offrant la possibilité de créer des backends puissants et des API.

De notre côté, on se penchera uniquement sur l'utilisation navigateurs web !

EXPLICATIONS

Sites web à explorer

Apple.com

Animations sophistiquées au scroll qui révèlent les produits de manière cinématographique. Un chef-d'œuvre de design interactif.

Bruno Simon

Un portfolio en 3D interactif où vous conduisez une voiture virtuelle. L'innovation JavaScript à son paroxysme !

CodePen Trending

Des centaines de mini-projets créatifs réalisés par la communauté. Une source d'inspiration infinie pour apprendre.

EXPLICATIONS

Interactions JavaScript sur Amazon

Recherche en temps réel

Les suggestions apparaissent instantanément pendant la saisie, améliorant la rapidité de la recherche.

Ajout au panier dynamique

Les articles sont ajoutés au panier sans recharger la page, offrant une transition rapide et sans accroc.

Carrousel d'images produits

Naviguez fluidement entre les différentes vues des produits grâce à un carrousel interactif.

Filtres de recherche dynamiques

Affinez les résultats de recherche en temps réel, sans aucun chargement de page.

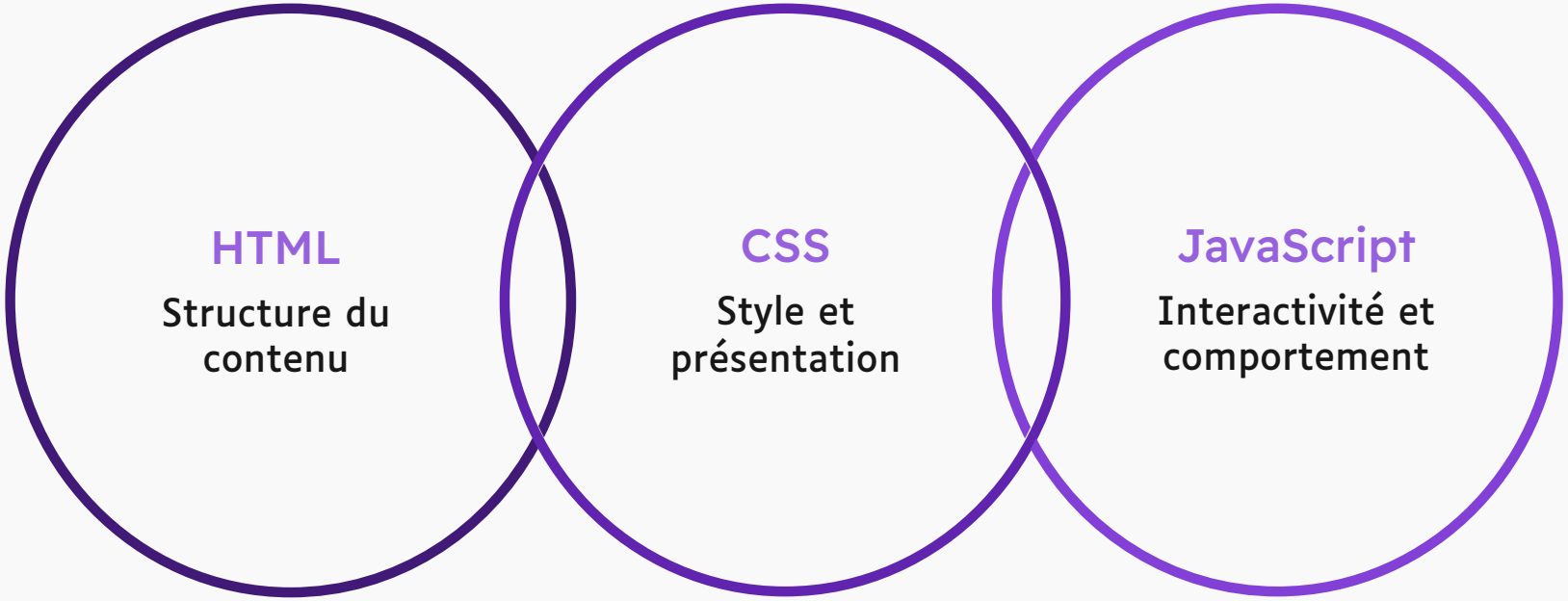
EXPLICATIONS

Décryptage des interactions

Interaction	Ce qui se passe	Élément HTML	Rôle de JavaScript
Clic "Ajouter au panier"	Le nombre change	Compteur	Modifier le texte
Survol d'une image	Zoom apparaît	Image	Afficher élément caché
Remplir un formulaire	Message d'erreur	Input email	Vérifier le format
Charger plus	Nouveaux produits	Liste	Ajouter du contenu

EXPLICATIONS

La trinité du web : HTML, CSS et JavaScript



Ces trois langages fondamentaux travaillent en synergie pour construire des expériences web riches et complètes. Chacun a un rôle distinct mais complémentaire, essentiel à la création d'une page web moderne.

EXPLICATIONS

Du pseudo-code au JavaScript

Comprendre la logique avant la syntaxe est essentiel. Voici un exemple simple de la façon dont une idée exprimée en pseudo-code se traduit en JavaScript pour afficher un message.

Pseudo-code

Le pseudo-code est une description informelle et de haut niveau de l'algorithme d'un programme.

```
Début
  Afficher "Bonjour tout le monde"
Fin
```

JavaScript

En JavaScript, la méthode `console.log()` est fréquemment utilisée pour afficher des informations ou des messages dans la console de développement du navigateur, un outil indispensable pour le débogage.

```
console.log("Bonjour tout le monde");
```

Mettons de côté les définitions et testons un peu tout ça !

OUTILS

Ouvrir les outils de développement (DevTools)

Les outils de développement web sont une suite d'outils intégrés aux navigateurs. Ils sont essentiels pour inspecter, modifier et débugger les pages web en temps réel.

Comment les ouvrir ?

Vous pouvez les ouvrir de plusieurs manières, selon votre navigateur :

- Faites un clic droit n'importe où sur la page et sélectionnez **"Inspecter"** ou **"Inspect Element"**.
- Utilisez le raccourci clavier : `F12` (Windows/Linux) ou `Cmd + Option + I` (macOS).

OUTILS

Les onglets principaux des DevTools

Onglets principaux à connaître :

Elements

Inspectez et modifiez la structure HTML, ainsi que les styles CSS appliqués à chaque élément.

Console

Affichez les messages de log, exécutez du JavaScript en direct et débuggez les erreurs qui surviennent.

Sources

Visualisez, modifiez et débuggez le code JavaScript de votre page avec des points d'arrêt.

Network

Analysez les requêtes réseau (chargement de ressources, appels d'API) et leur performance.

Commençons par nous rendre dans la console pour les premiers pas avec JavaScript.

MISE EN PRATIQUE

Utilisation de la console

La méthode `console.log()` est votre premier outil pour interagir avec le monde de JavaScript. Elle vous permet d'afficher des informations et de visualiser ce qui se passe dans votre code directement dans la console de votre navigateur.

Comment s'exercer ?

1. Ouvrez les **DevTools** de votre navigateur.
2. Sélectionnez l'onglet **"Console"**.
3. Tapez les lignes de code ci-contre, une par une, et appuyez sur **Entrée** après chacune.
4. Observez attentivement le résultat affiché dans la console après chaque exécution.

Code à tester dans la console

```
console.log("Bonjour le monde !");
```

```
console.log(123);
```

```
console.log(10 + 5 * 2);
```

Cette méthode est essentielle pour le debug et pour comprendre le flux d'exécution de vos scripts JavaScript. Prenez le temps de jouer avec différents messages et calculs !

OUTILS

Autres méthodes de la console

En plus de `console.log()`, la console de votre navigateur offre d'autres fonctions pratiques pour le debug.

`console.error()`

Affiche un message d'erreur dans la console, souvent en rouge, pour signaler un problème.

`console.warn()`

Utilisé pour afficher des avertissements. Ces messages sont généralement mis en évidence en jaune.

`console.info()`

Pour afficher des messages d'information généraux, souvent accompagnés d'une petite icône "i" ou d'une couleur spécifique.

`console.clear()`

Efface tout le contenu précédemment affiché dans la console, utile pour repartir d'une console propre lors du debugging.

Il en existe encore plein d'autres, ce sera à vous de les découvrir par la suite !

MODULE

Déclarer et utiliser des variables

EXPLICATIONS

Qu'est-ce qu'une variable ?

En JavaScript (et dans la plupart des langages de programmation), une variable est un **conteneur nommé** utilisé pour **stocker des données**.

Ces données peuvent être du texte, des nombres, des listes, ou des objets plus complexes. La capacité de faire varier ces valeurs est fondamentale pour créer des applications dynamiques et interactives.

Imaginez une variable comme une boîte étiquetée

Vous lui donnez un nom, et vous pouvez y placer n'importe quelle valeur. L'avantage principal est que le contenu de cette boîte peut être modifié à tout moment pendant l'exécution de votre programme.

EXPLICATIONS

L'anatomie d'une variable

Une variable passe par différentes phases dans son cycle de vie : sa déclaration, où elle est introduite, et son assignation, où une valeur lui est donnée. Cette valeur peut ensuite être modifiée.

01

Déclaration

Le mot-clé `let` est utilisé pour déclarer une variable, signalant au programme qu'un espace mémoire sera réservé.

```
let maVariable;
```

02

Assignation (initiale)

L'opérateur d'assignation (`=`) est utilisé pour donner une première valeur à la variable.

```
maVariable = "Bonjour";
```

03

Modification (Ré-assignation)

Après son assignation initiale, la valeur d'une variable déclarée avec `let` peut être changée à tout moment.

```
maVariable = "Au revoir";
```

EXPLICATIONS

Le trio des variables

Le choix du mot-clé pour déclarer une variable est essentiel en JavaScript.
Comprendre ces différences est fondamental pour écrire un code propre et prévisible.

1
var

2
let

3
const

EXEMPLE

Exemple d'utilisation de var

Le mot-clé `var`, bien que moins utilisé aujourd'hui, était la façon standard de déclarer des variables en JavaScript.

```
// Déclaration et initialisation
var nom = "Alice";
console.log(nom); // Affiche : Alice

// Re-assignation (autorisée avec var)
nom = "Marie";
console.log(nom); // Affiche : Marie

// Re-déclaration (autorisée avec var)
var nom = "Bob";
console.log(nom); // Affiche : Bob
```

EXEMPLE

Exemple d'utilisation de const

Le mot-clé `const` est utilisé pour déclarer des constantes. Une fois qu'une valeur est assignée à une variable déclarée avec `const`, elle ne peut plus être modifiée.

```
// Déclaration et initialisation d'une constante
const PI = 3.14159;
console.log(PI); // Affiche : 3.14159

// Tentative de re-assignation (erreur)
PI = 3.14; // Uncaught TypeError: invalid assignment to const 'PI'.

// Tentative de re-déclaration (erreur)
const PI = 3.141; // Uncaught SyntaxError: redeclaration of const PI
```

EXEMPLE

Exemple d'utilisation de let

Le mot-clé `let` est le choix recommandé pour déclarer des variables dont la valeur est susceptible de changer.

```
// Déclaration et initialisation
let age = 30;
console.log(age); // Affiche : 30

// Re-assignation (autorisée)
age = 31;
console.log(age); // Affiche : 31

// Tentative de re-déclaration dans le même scope (erreur)
let age = 32; // Uncaught SyntaxError: redeclaration of let age
```

EXPLICATIONS

let vs const: Le choix judicieux

En JavaScript, le choix entre `let` et `const` dépend de la mutabilité de la variable que vous souhaitez déclarer.

`let`

- Déclare une variable dont la valeur **peut être réaffectée**.
- Possède une **portée de bloc** (block scope).
- À utiliser lorsque la valeur de la variable est susceptible de changer au cours de l'exécution du programme.

`const`

- Déclare une constante dont la valeur **ne peut pas être réaffectée** après son initialisation.
- Possède également une **portée de bloc** (block scope).
- Idéal pour les valeurs qui ne doivent jamais changer, garantissant une meilleure prévisibilité du code.

Utilisez toujours `const` par défaut. Ne passez à `let` que si vous avez une raison spécifique de réaffecter la variable. Oubliez `var` pour vos nouveaux projets !

EXPLICATIONS

Pourquoi éviter var ?

Bien que fonctionnel, l'utilisation de `var` peut introduire des comportements inattendus et des bugs difficiles à détecter en raison de sa portée ("scope") et de son mécanisme de "hoisting". Ces particularités peuvent rendre le code moins prévisible.

⚠ Pour un code plus robuste, lisible et moins source d'erreurs, il est **fortement recommandé** d'utiliser `let` ou `const` pour vos déclarations de variables.

MODULE

Identifier et manipuler les types de données

EXPLICATIONS

Les types de données primitifs en JavaScript

JavaScript manipule différents types de données pour stocker et traiter l'information. Les types primitifs sont les blocs de construction fondamentaux du langage.

01

String

Utilisé pour représenter du texte (séquences de caractères).

02

Number

Représente des nombres entiers ou décimaux.

03

Boolean

Représente une valeur logique, vrai ou faux.

04

Undefined

Indique qu'une variable a été déclarée mais n'a pas encore de valeur assignée.

05

Null

Représente l'absence intentionnelle de toute valeur.

EXPLICATIONS

Le type String (Chaîne de caractères)

Le type `String` est utilisé pour représenter du texte en JavaScript. Une chaîne de caractères est une séquence de caractères enveloppée entre guillemets simples ou doubles.

```
const firstname = "Alice";
const lastname = 'Dupont'; // Guillemets simples ou doubles
const greeting = "Bonjour tout le monde !";
```

EXEMPLE

Concaténation de chaînes de caractères

La concaténation est l'opération qui permet de combiner plusieurs chaînes de caractères, ou des chaînes et des variables, en une seule chaîne. En JavaScript, il existe deux méthodes principales pour y parvenir.

1. L'opérateur d'addition (`+`)

C'est la méthode traditionnelle pour assembler des chaînes. Vous placez le signe plus entre chaque élément à combiner.

```
const nom = "Alice";
const message = "Bonjour " + nom;

console.log(message);
// Affiche : Bonjour Alice
```

2. Les littéraux de gabarits (Template Literals)

Les littéraux de gabarits utilisent les accents graves (backticks) et permettent d'intégrer des expressions directement dans la chaîne.

```
const nom = "Bob";
const message = `Bonjour ${nom}`;

console.log(message);
// Affiche : Bonjour Bob
```

EXPLICATIONS

Le type Number (Nombre)

Le type `Number` est fondamental en JavaScript pour représenter toute forme de valeur numérique, qu'il s'agisse d'entiers, de nombres à virgule flottante, ou même de valeurs spéciales comme `Infinity` ou `NaN` (Not a Number).

```
const age = 25; // Un entier
const price = 19.99; // Un nombre à virgule flottante
const temperature = -5; // Un nombre négatif
const bigNumber = 123e5; // Notation scientifique (12300000)
const smallNumber = 1e-3; // Notation scientifique (0.001)
```

EXEMPLE

Opérations mathématiques

Addition

```
const sum = 5 + 3; // 8
```

Soustraction

```
const sub = 10 - 4; // 6
```

Multiplication

```
const mult = 6 * 7; // 42
```

Division

```
const divide = 20 / 4; // 5
```

EXPLICATIONS

Le type Boolean (Booléen)

Le type `Boolean` représente une valeur logique qui peut être soit `true` (vrai) soit `false` (faux). Il est fondamental pour les opérations de contrôle de flux et les prises de décision dans le code.

```
const isAdmin = true;
const hasLicense = false;
```

EXEMPLE

Comparaisons et booléens

Égalité

```
5 === 5      // true
5 === 10     // false
```

Inégalité

```
5 !== 10    // true
5 !== 5     // false
```

Supérieur

```
10 > 5      // true
5 > 10      // false
```

Inférieur

```
3 < 7       // true
7 < 3       // false
```

EXPLICATIONS

Égalité stricte ou non stricte ?

`==` (Égalité non stricte)

Cette opération compare les **valeurs** après avoir effectué des conversions de type implicites. Elle peut produire des résultats surprenants et peu intuitifs.

```
'5' == 5   // true ('5' converti en nombre)
0 == false // true (0 converti en booléen)
null == undefined // true
```

`===` (Égalité stricte)

Cette opération compare les **valeurs ET les types** sans aucune conversion implicite. Elle offre un comportement prévisible et fiable.

```
'5' === 5          // false
0 === false        // false
null === undefined     // false
```

Recommandation importante

Pour la clarté et la prévention des bugs, utilisez `===` (égalité stricte) **systématiquement** dans votre code JavaScript. C'est une bonne pratique fondamentale !

EXPLICATIONS

Les types Undefined et Null

Au-delà des types classiques, JavaScript possède des valeurs spéciales pour représenter l'absence ou l'indétermination.

`Undefined` (Non défini)

Indique qu'une variable a été déclarée mais n'a pas encore de valeur assignée. C'est une absence de valeur par défaut.

```
let result;
console.log(result); // Affiche : undefined
```

`Null` (Valeur nulle)

Représente l'absence intentionnelle de toute valeur. C'est une valeur qu'un développeur assigne explicitement pour signifier "rien".

```
const data = null; // Valeur intentionnellement vide
console.log(data); // Affiche : null
```

EXPLICATIONS

Vérifier le type d'une variable

En JavaScript, l'opérateur `typeof` est un outil utilisé pour déterminer le type de données d'une variable ou d'une expression. Il renvoie une chaîne de caractères indiquant le type.

```
const name = "Alice";
const age = 30;
const isStudent = true;

console.log(typeof name);      // "string"
console.log(typeof age);       // "number"
console.log(typeof isStudent); // "boolean"
```

QUESTION OUVERTE

Que vaut '1' + 1 ?

Prenez un instant pour réfléchir à l'opérateur d'addition en JavaScript et à la façon dont il interagit avec les chaînes de caractères et les nombres.

Vous avez évidemment le droit d'essayer dans la console !

EXPLICATION

Pourquoi '1' + 1 = '11' et pas 2 ?

Lorsque l'opérateur d'addition (`+`) est utilisé avec une chaîne de caractères et un nombre, JavaScript effectue un processus appelé coercition de type.

Dans ce scénario, le nombre est converti en chaîne de caractères avant l'opération.

```
const result = '1' + 1;
console.log(result); // Affiche : "11"
console.log(typeof result); // Affiche : "string"
```

Une fois les deux opérandes sous forme de chaînes de caractères, l'opérateur `+` effectue une concaténation de chaînes, c'est-à-dire qu'il les assemble côte à côte.

> C'est une spécificité de JavaScript. Avec d'autres opérateurs comme `-`, `*` ou `/`, JavaScript tenterait de convertir la chaîne en nombre (si possible) pour effectuer une opération arithmétique.

EXPLICATIONS

Convertisseurs de types

JavaScript offre des fonctions pour convertir explicitement une valeur d'un type à un autre.

Vers un Nombre

- `Number(value)`: Convertit toute valeur en type `Number`.
- `parseInt(string, radix)`: Parse une chaîne et retourne un entier.
- `parseFloat(string)`: Parse une chaîne et retourne un nombre à virgule flottante.

Vers une Chaîne

- `String(value)`: Convertit n'importe quelle valeur (nombre, booléen, etc.) en sa représentation textuelle.
- `value.toString()`: Méthode disponible sur la plupart des types de données pour les convertir en chaîne.

Et si on passait du langage naturel au code ?

QUESTION OUVERTE

Que peut-on comprendre de cette phrase ?

Considérez les informations contenues dans la phrase suivante et identifiez ce qu'il faut mettre en place en JavaScript pour pouvoir représenter cette phrase en code.

> Je m'appelle Quentin et j'ai 22 ans.

Quels sont les différents "morceaux" d'information et comment JavaScript les interpréterait-il ?

EXPLICATIONS

Traduire la phrase en code JavaScript

1

`const firstName`

Pour stocker le prénom "Quentin", qui est une chaîne de caractères, nous utilisons le type **String**.

```
const firstName = "Quentin";
```

2

`const age`

Pour stocker l'âge 22, qui est une valeur numérique, nous utilisons le type **Number**.

```
const age = 22;
```

EXEMPLE

Plus d'exemples de traduction

Phrase naturelle	Code JavaScript
"Le prix est de 29 euros"	`const price = 29;`
"L'utilisateur est connecté"	`const isConnected = true;`
"Mon email est alex@mail.com"	`const mailAddress = "alex@mail.com";`
"Il y a 150 étudiants"	`const studentsCount = 150;`

MISE EN PRATIQUE

Déclarez vos premières variables

Mettez en pratique ce que nous avons vu en déclarant des variables pour stocker des informations personnelles et affichez-les dans la console de votre navigateur.

Étapes à réaliser :

Déclarer des variables

- Prénom
- Age
- Ville

Afficher dans la console

Afficher une phrase de présentation, comme :

"Bonjour, je m'appelle Alice, j'ai 30 ans et j'habite à Orléans."

N'hésitez pas à expérimenter en changeant les valeurs et en observant les résultats.

MODULE

Conseils sur la propreté de code

CONSEILS

Pourquoi avoir un code propre est important

Vouloir un code propre n'est pas qu'une question d'esthétique, c'est un pilier fondamental pour le développement logiciel durable et efficace.

Lisibilité & Maintenabilité

Un code propre est facile à lire et à comprendre, facilitant les futures modifications et la correction des bugs.

Collaboration Améliorée

Des conventions claires permettent aux équipes de travailler ensemble de manière plus fluide et efficace.

Réduction des Bugs

Un code clair et bien structuré est moins sujet aux erreurs, augmentant la fiabilité de votre application.

CONSEILS

Nommer vos variables intelligemment

Bonnes pratiques

- `firstName`
- `totalOrders`
- `isConnected`
- `messagesCount`

À éviter

- `x` (trop vague)
- `data` (pas spécifique)
- `temp` (peu descriptif)
- `var1` (sans contexte)

Un bon nom de variable explique son contenu et son utilité !

CONSEILS

Conventions de nommage

camelCase

La norme JavaScript : `monNomDeVariable`.
Première lettre minuscule, puis majuscules pour chaque mot.

Descriptif et précis

Privilégiez la clarté : `prixTotalTTC` plutôt que `p` ou `prix`.

Éviter les accents

Préférez `prenom` à `prénom` pour éviter les problèmes d'encodage.

⚠ Même si coder en français est totalement autorisé, privilégiez la langue anglaise dans votre code pour vous familiariser avec la langue et pouvoir travailler plus facilement avec n'importe quel autre développeur !

CONSEILS

Les commentaires : Documenter votre code

```
// Commentaire sur une seule ligne
const productPrice = 29.99; // Prix en euros

/*
  Commentaire sur
  plusieurs lignes
  pour des explications détaillées
*/
const productDiscount = 0.15; // 15% de réduction
```

Les commentaires aident les autres développeurs (et vous-même dans le futur) à comprendre votre code.

CONSEILS

Quand commenter ?

Expliquer le pourquoi, pas le comment

Commentez les décisions importantes et la logique métier, pas chaque ligne de code.

Clarifier les calculs complexes

Si une formule ou un calcul n'est pas évident, ajoutez un commentaire explicatif.

Marquer les zones à améliorer

Utilisez `// TODO:` ou `// FIXME:` pour les tâches futures.

⚠ Attention à ne pas tomber dans le piège des "TODO" et compagnie !

MODULE

Interagir avec l'utilisateur

EXPLICATIONS

Pourquoi interagir avec l'utilisateur ?

Un programme JavaScript prend toute sa valeur lorsqu'il va au-delà de la simple exécution de code, en impliquant activement l'utilisateur.

Il existe plusieurs manières d'interagir avec l'utilisateur, de notre côté on va aborder les fenêtres modales natives au navigateur.

EXEMPLE

Alert : Afficher un message

```
alert("Bienvenue sur mon site !");
alert("Attention : offre limitée");
```

La fonction `alert()` affiche une boîte de dialogue avec votre message.

EXEMPLE

Prompt : Demander une information

```
// Demander le nom de l'utilisateur
const firstName = prompt("Comment vous appelez-vous ?");
console.log(firstName);
```

La fonction `prompt()` affiche une boîte de dialogue avec un message. Cette boîte de dialogue contient un input permettant à l'utilisateur de transmettre du texte.

MISE EN PRATIQUE

Exercice guidé : Mini-questionnaire

Vous vous souvenez de la mise en pratique précédente, avec les déclarations de variables ?

Il faudra modifier le code final pour demander à l'utilisateur avec des `prompt()` ses informations puis les afficher avec la fonction `alert()` !

EXEMPLE

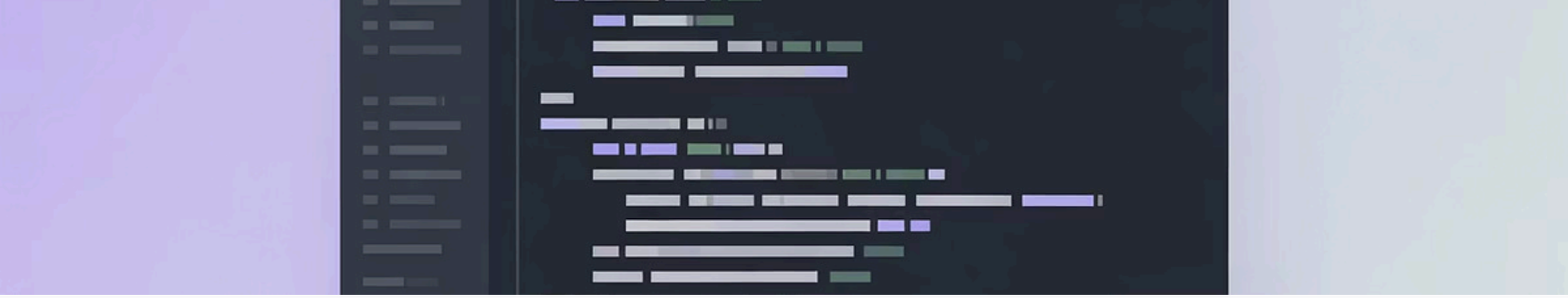
Corrigé : Mini-questionnaire

```
const firstName = prompt("Votre prénom ?");
const age = prompt("Votre âge ?");
const city = prompt("Votre ville ?");

const message = `Bonjour ${firstName}, ${age} ans, de ${city} !`;
alert(message);
```

Avant de continuer…

Rendons les choses plus simples et plus propres !



Créons un mini-projet

Intégrons HTML et JavaScript en utilisant la balise `<script>` pour voir le code en action.

Étapes

1

Création d'un dossier

Créez un répertoire de travail pour cette journée et ouvrez-le sur votre éditeur de code.

2

Création du HTML

Générez la base HTML (html, head et body) dans un fichier `index.html`.

3

Insertion du JavaScript

Dans le fichier HTML, ajoutez juste avant la fermeture du body une balise `<script>` avec un `console.log()` dedans.

ⓘ Ouvrez ensuite votre fichier index.html dans un navigateur pour vérifier si tout fonctionne !

MISE EN PRATIQUE

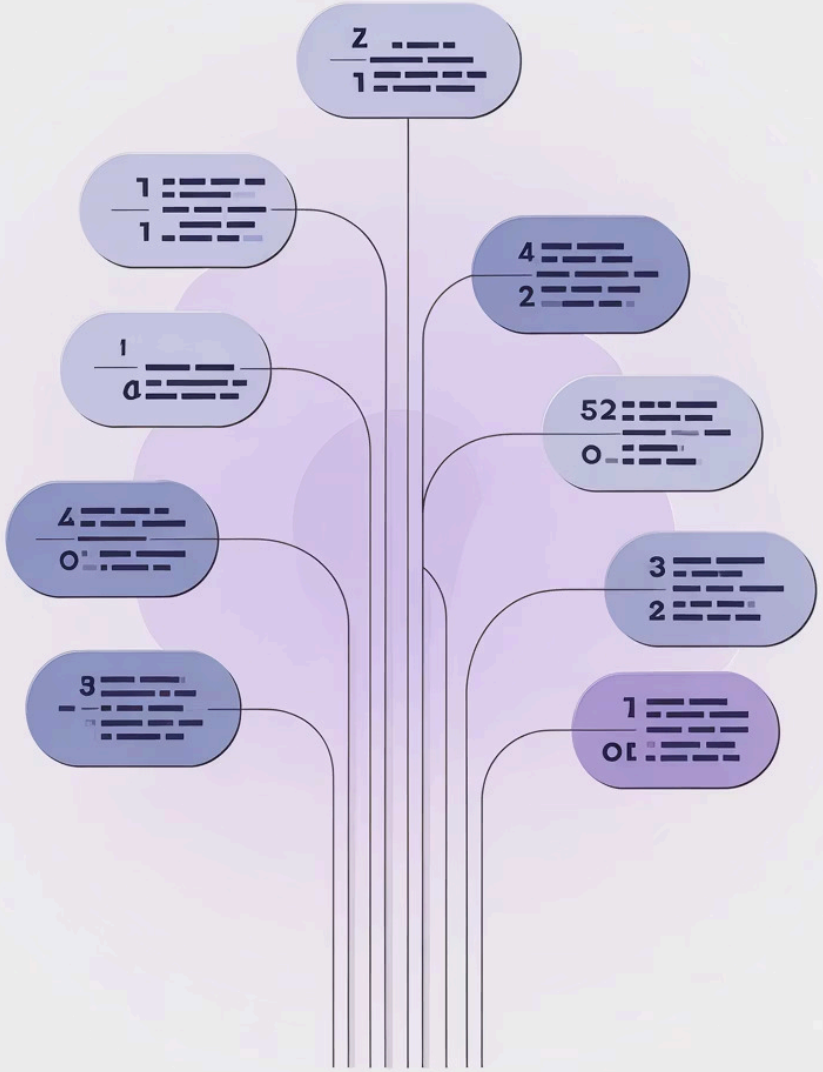
Calculateur de pourboire

Cette fois-ci, on va faire quelque chose d'un peu plus utile même si ça reste très basique.

Il va vous falloir créer un calculateur interactif qui demande le montant de l'addition et le pourcentage de pourboire. Ensuite, calculez le montant du pourboire puis affichez le montant total.

MODULE

Scénarios d'exécutions



EXPLICATIONS

Les décisions du quotidien comme des conditions

Dans la vie de tous les jours, nous prenons constamment des décisions basées sur des conditions. C'est exactement le principe des conditions en JavaScript : exécuter un code spécifique SI une certaine condition est remplie.

S'il pleut...

... je prends mon parapluie.

Si j'ai faim...

... je mange.

Si le feu est rouge...

... je m'arrête.

Chacune de ces situations illustre une condition simple : un "si" (`if`) suivi d'une action.

EXEMPLE

Exemple avec Amazon

Panier vide

- Message "Votre panier est vide"
- Bouton "Commander" désactivé
- Aucune option de paiement

Panier rempli

- Affichage du total
- SI total > 50€ → "Livraison gratuite"
- SINON → "Livraison 5,99€"

EXPLICATIONS

Le concept fondamental

1

La question

SI [quelque chose est vrai]

2

L'action principale

ALORS [faire cette action]

3

L'alternative

SINON [faire cette autre action]

EXEMPLE

Exemple concret avec la météo

Condition	Action si vrai	Action si faux
Il pleut ?	Prendre un parapluie	Ne rien prendre

EXPLICATIONS

Vocabulaire clé à retenir

Condition

La question qu'on pose

Exemple : il pleut ?

Bloc d'instruction

Ce qu'on fait si c'est vrai

Exemple : prendre parapluie

Alternative

Ce qu'on fait si c'est faux

Exemple : ne rien prendre

EXEMPLE

Exemple : La température

01

En langage naturel

Si la température est supérieure à 25, alors afficher "Il fait chaud"

02

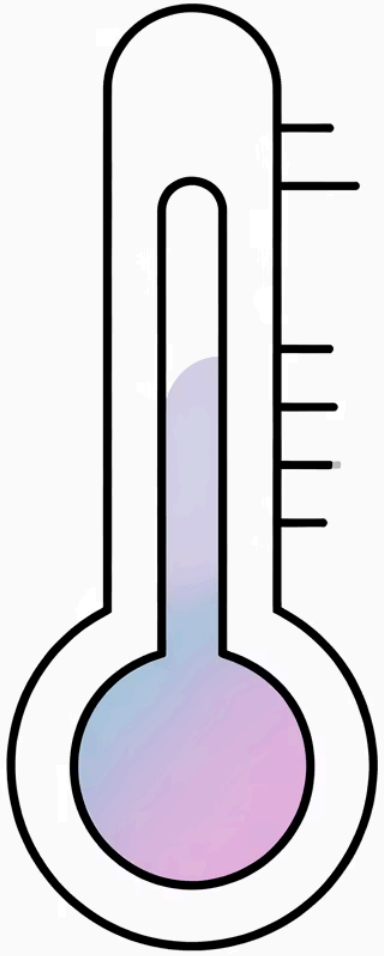
En pseudo-code

```
SI temperature > 25 ALORS
  Afficher "Il fait chaud"
FIN SI
```

03

En JavaScript

```
if (temperature > 25) {
  console.log("Il fait chaud");
}
```



EXPLICATIONS

Décortiquage de la syntaxe

1	**Le mot-clé** `if` est le mot magique JavaScript pour "si"
2	**Les parenthèses** `(condition)` sont **OBLIGATOIRES** autour de la condition
3	**Les accolades** `{}` délimitent le bloc d'instructions à exécuter
4	**L'indentation** 2 espaces pour la lisibilité du code

EXEMPLE

Version complète avec trois cas

```
const temperature = 28;

if (temperature > 25) {
 console.log("Il fait chaud");
} else if (temperature > 15) {
 console.log("Il fait bon");
} else {
 console.log("Il fait froid");
}

// Résultat : "Il fait chaud"
```

`else if` **permet d'appliquer une nouvelle condition à la suite de la précédente, uniquement si la condition précédente ne valait pas** `true`**.**

`else` **permet d'appliquer un comportement par défaut si aucune des conditions du dessus ne valait** `true`**.**

EXPLICATIONS

Les opérateurs de comparaison

Opérateur	Signifie	Exemple
`===`	égal à	`age === 18`
`!==`	différent de	`name !== "Admin"`
`>`	supérieur à	`price > 100`
`<`	inférieur à	`stock < 10`
`>=`	supérieur ou égal	`age >= 18`
`<=`	inférieur ou égal	`grade <= 20`

⚠ Attention à ne pas utiliser `==` ou `!=` dans votre code ! Ces opérateurs ne sont pas stricts et peuvent être à l'origine de bugs dans l'interprétation.

CONSEILS

Le piège classique à éviter

FAUX - Affectation

```
if (age = 18) {
  // Ici on ASSIGNE 18 à age !
  console.log("Majeur");
}
```

Un seul `=` modifie la variable

CORRECT - Comparaison

```
if (age === 18) {
  // Ici on TESTE si age vaut 18
  console.log("Majeur");
}
```

Trois `===` pour comparer les valeurs

MISE EN PRATIQUE

Contrôle d'accès

En reprenant votre code précédent, mettez en place un système de contrôle d'accès simple qui n'autorisera l'entrée qu'aux personnes majeures.

1

Condition à vérifier

L'âge doit être supérieur ou égal à **18 ans**.

2

Si VRAI

Autoriser l'entrée et afficher un message de bienvenue.

3

Si FAUX

Refuser l'entrée et expliquer clairement la raison.

Bonus pour les plus rapides

Pourquoi pas ajouter un espace VIP réservé exclusivement aux personnes qui ont entre 20 et 24 ans ?

EXPLICATIONS

Les opérateurs logiques

Opérateur	Signifie	Exemple
`&&`	ET	`age >= 20 && age <= 24`
`||`	OU	`isAdmin || isModerator`
`!`	NON	`!isBlocked`

Ces opérateurs permettent de combiner plusieurs conditions !

EXPLICATIONS

Opérateur ET (&&)

```
// Les DEUX conditions doivent être vraies
const age = 20;
const hasIDCard = true;

if (age >= 18 && hasIDCard) {
  console.log("Peut voter");
}
```

Les **deux** côtés du && doivent être vrais pour que le bloc s'exécute.

EXPLICATIONS

Opérateur OU (||)

```
// AU MOINS UNE condition doit être vraie
let isAdmin = false;
let isModerator = true;

if (isAdmin || isModerator) {
  console.log("Peut modérer le contenu");
}
```

Il suffit qu'**un seul** côté du || soit vrai pour exécuter le bloc.

EXPLICATIONS

Opérateur NON (!)

```
// Inverse le booléen
let isBlocked = false;

if (!isBlocked) {
  console.log("Peut se connecter");
}
```

Le `!` transforme `true` en `false` et vice versa.

CONCLUSION

Et… on s'arrête là pour les notions du jour !

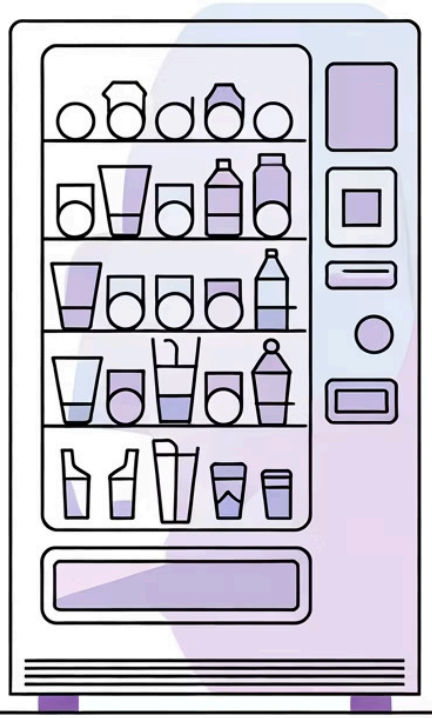
Ça fait beaucoup (vraiment), donc on fait une pause pour aujourd'hui.

… mais ce n'est pas tout à fait fini 😉

MISE EN PRATIQUE

Distributeur automatique

Programmez la logique complète d'un distributeur de boissons.



Catalogue de produits

1€	1.50€	2€	1.20€
💧 Eau	🥤 Soda	🧃 Jus	☕ Café

Spécifications du système

01

Insertion d'argent

Le client insère de l'argent dans la machine

02

Choix du produit

Le client sélectionne le produit désiré

03

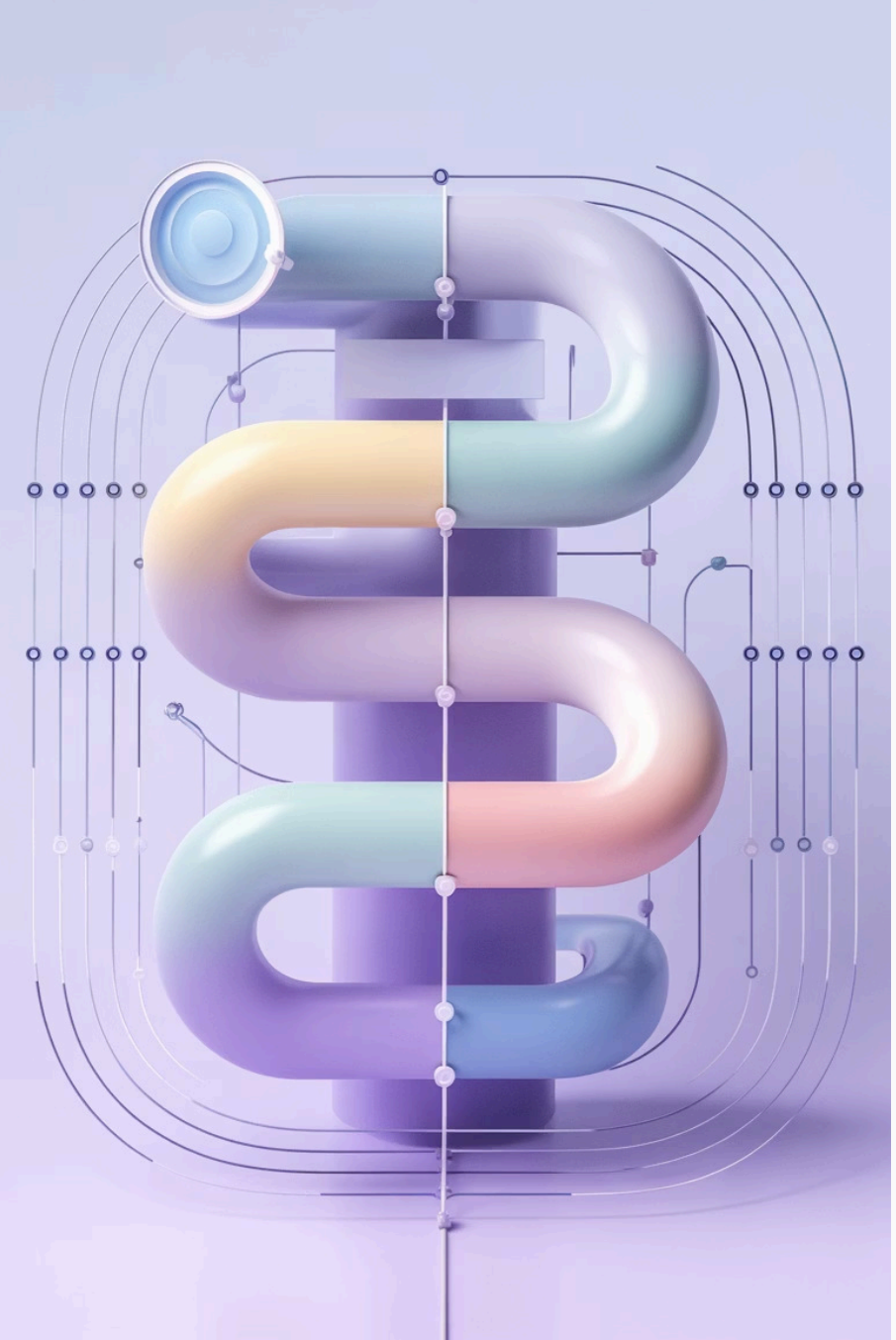
Vérifications

Le produit existe ? Argent suffisant ?

04

Distribution

Distribue le produit et rend la monnaie



À SUIVRE

Prochaines étapes

Demain, nous nous retrouverons pour explorer deux concepts qui rendront vos programmes beaucoup plus dynamiques : les **structures de données** et les **boucles**.

Et pas de panique, ce qu'on a vu aujourd'hui sera approfondi au fur et à mesure !